

Machine Coding Round – Complete Guide for 80 LPA Interviews

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 12 Dec 2026 – 3 Jan 2027 (Days 118-140)

 **Your slots (IST):** Mon-Fri **7:30-8:30pm & 9:00-10:00pm** · Sat-Sun **9:00am-12:00pm, 3:00-5:30pm, 6:00-8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import [calendar/mern-mastery-study.ics](https://calendar.google.com/calendar/ical/harsh.gupta%40getreelax.com/public/calendar/mern-mastery-study.ics)

90-minute machine coding rounds at Google L5, Meta E5, Amazon L6, and top Indian product companies (80+ LPA). Full implementations, SOLID principles, and evaluation criteria.

Table of Contents

1. [What Is Machine Coding at 80 LPA Companies?](#)
2. [The 90-Minute Approach Framework](#)
3. [Problem 1: Parking Lot \(Multi-Floor, Multiple Vehicle Types\)](#)
4. [Problem 2: In-Memory Rate Limiter \(Sliding Window\)](#)
5. [Problem 3: LRU Cache](#)
6. [Problem 4: Snake & Ladder](#)
7. [Problem 5: Splitwise Expense Splitting](#)
8. [Problem 6: BookMyShow Seat Booking with Locking](#)
9. [Problem 7: Elevator System](#)
10. [Problem 8: Vending Machine \(State Pattern\)](#)
11. [Evaluation Criteria Interviewers Use](#)
12. [10 Machine Coding Interview Q&A](#)
13. [Frontend machine coding \(1 CR\)](#)

1. What Is Machine Coding at 80 LPA Companies?

Definition

Machine coding (also called **Low-Level Design + Live Coding**) is a 60–90 minute round where you build a **working, runnable system** from scratch — not pseudocode, not just class diagrams. You write real code (TypeScript/Java/Python) that handles core flows, edge cases, and extensibility.

Where It Appears

Company Tier	Round Name	Duration	Expectation
Google L5	LLD / Coding Design	45–60 min	Clean OOP, extensible design
Meta E5	Product Architecture	60 min	Working code + trade-offs
Amazon L6	OOD + Coding	90 min	SOLID, patterns, test cases
Flipkart SDE-3	Machine Coding	90 min	Full working system
Razorpay / Swiggy / PhonePe	LLD Round	60–90 min	Production-quality code
Uber / Goldman	OOD Round	60 min	Design + implement core path

What Makes 80 LPA Different from 20 LPA

At 80 LPA (roughly Google L5 / Meta E5 / Staff at Indian unicorns), interviewers expect:

1. **Requirements clarification** — You ask 5–10 sharp questions before coding
2. **API-first design** — Public interfaces defined before internals
3. **Working code** — At least the happy path runs without bugs
4. **SOLID + patterns** — Not over-engineered, but clearly applied
5. **Edge cases** — Null checks, concurrency awareness, overflow
6. **Extensibility** — "What if we add a new vehicle type?" answered in design

7. **Communication** — Think aloud; interviewer is a collaborator
8. **Testing mindset** — Mention or write 2–3 test cases

Common Problems (Frequency at 80 LPA)

Problem	Frequency	Key Pattern
Parking Lot	Very High	Strategy + Factory
Rate Limiter	Very High	Sliding Window / Token Bucket
LRU Cache	High	HashMap + Doubly Linked List
Splitwise	High	Graph / Balance Sheet
BookMyShow	High	Locking / Seat Map
Snake & Ladder	Medium	State Machine
Elevator	Medium	Scheduler / Strategy
Vending Machine	Medium	State Pattern
Tic-Tac-Toe	Medium	Strategy
Chess (basic)	Low	Command Pattern

2. The 90-Minute Approach Framework

Phase 1: Requirements (0–10 min)

ASK:

- └ Functional: What operations? Who are the actors?
- └ Non-functional: Scale? Concurrency? Persistence?
- └ Scope: What's IN vs OUT for this round?
- └ Edge cases: Empty input? Duplicate? Overflow?
- └ API shape: REST endpoints or class methods?

Template questions:

- "Should this be in-memory or persisted?"
- "Do we need thread safety / concurrent access?"

- "What's the expected scale — single machine or distributed?"
- "Can I assume a single user or multi-user?"
- "Should I prioritize correctness or also discuss scaling?"

Phase 2: API Design (10-20 min)

Draw or write public interfaces first:

```
// Example: Parking Lot API surface
interface ParkingLotService {
  parkVehicle(vehicle: Vehicle): Ticket | null;
  unparkVehicle(ticket: Ticket): number; // returns fee
  getAvailableSpots(floor?: number): SpotCount;
}
```

Rules:

- Define enums (`VehicleType` , `SpotType`) before classes
- Use interfaces for abstractions (`PricingStrategy` , `SpotAllocator`)
- Name methods as verbs: `bookSeat` , `releaseSeat` , `calculateFee`

Phase 3: Code (20-70 min)

Priority order:

1. Core entities (Vehicle, Spot, Ticket)
2. Main flow (park → assign spot → return ticket)
3. Secondary flows (unpark, fee calculation)
4. Edge cases (lot full, invalid ticket)
5. Extensibility hooks (new vehicle type)

Coding discipline:

- Start with types/enums
- One class per file mentally (even if one buffer)
- No premature optimization
- Use `Map` / `Set` over arrays for O(1) lookup

Phase 4: Test (70-85 min)

```
// Always mention or write:
describe('ParkingLot', () => {
  it('parks car when spot available', () => { /* ... */ });
  it('returns null when lot full', () => { /* ... */ });
  it('calculates correct fee on unpark', () => { /* ... */ });
});
```

Phase 5: Trade-offs (85-90 min)

- "For production I'd add Redis for distributed locking"
- "I'd persist tickets to DB with optimistic locking"
- "Rate limiter would use Redis ZSET for multi-instance"

3. Problem 1: Parking Lot

Requirements

- Multi-floor parking lot
- Vehicle types: Motorcycle, Car, Truck (each needs different spot size)
- Operations: park, unpark, get availability
- Fee based on duration and vehicle type
- Return null / error when full

SOLID Mapping

Principle	Application
S	SpotAllocator only assigns spots; FeeCalculator only computes fees
O	New vehicle types via VehicleType enum + strategy without changing allocator
L	All spots implement ParkingSpot interface
I	Separate ReadableLot (availability) from WritableLot (park/unpark)
D	ParkingLotService depends on FeeStrategy interface, not concrete class

Full Implementation

```
enum VehicleType {
  MOTORCYCLE = 'MOTORCYCLE',
  CAR = 'CAR',
  TRUCK = 'TRUCK',
}

enum SpotType {
  SMALL = 'SMALL',    // motorcycle
  MEDIUM = 'MEDIUM', // car
  LARGE = 'LARGE',    // truck
}

interface Vehicle {
  id: string;
  type: VehicleType;
  licensePlate: string;
}

interface Ticket {
  id: string;
  vehicleId: string;
  spotId: string;
  floor: number;
  entryTime: Date;
}

interface ParkingSpot {
  id: string;
  floor: number;
  type: SpotType;
  isOccupied: boolean;
  park(vehicle: Vehicle): Ticket;
  unpark(): Vehicle | null;
}

class ConcreteSpot implements ParkingSpot {
  id: string;
  floor: number;
  type: SpotType;
  isOccupied = false;
  private vehicle: Vehicle | null = null;

  constructor(id: string, floor: number, type: SpotType) {
    this.id = id;
    this.floor = floor;
    this.type = type;
  }
}
```

```

    park(vehicle: Vehicle): Ticket {
        if (this.isOccupied) throw new Error('Spot already occupied');
        this.vehicle = vehicle;
        this.isOccupied = true;
        return {
            id: `TKT-${Date.now()}`,
            vehicleId: vehicle.id,
            spotId: this.id,
            floor: this.floor,
            entryTime: new Date(),
        };
    }

    unpark(): Vehicle | null {
        const v = this.vehicle;
        this.vehicle = null;
        this.isOccupied = false;
        return v;
    }
}

// Strategy: vehicle → required spot type
const VEHICLE_SPOT_MAP: Record<VehicleType, SpotType> = {
    [VehicleType.MOTORCYCLE]: SpotType.SMALL,
    [VehicleType.CAR]: SpotType.MEDIUM,
    [VehicleType.TRUCK]: SpotType.LARGE,
};

interface FeeStrategy {
    calculate(ticket: Ticket, exitTime: Date, vehicleType: VehicleType): number;
}

class HourlyFeeStrategy implements FeeStrategy {
    private rates: Record<VehicleType, number> = {
        [VehicleType.MOTORCYCLE]: 20,
        [VehicleType.CAR]: 40,
        [VehicleType.TRUCK]: 60,
    };

    calculate(ticket: Ticket, exitTime: Date, vehicleType: VehicleType): number {
        const hours = Math.ceil(
            (exitTime.getTime() - ticket.entryTime.getTime()) / (1000 * 60 * 60)
        );
        return hours * this.rates[vehicleType];
    }
}

class SpotAllocator {
    constructor(private spots: ParkingSpot[]) {}

    findSpot(vehicleType: VehicleType): ParkingSpot | null {
        const required = VEHICLE_SPOT_MAP[vehicleType];
    }
}

```

```

// Truck can use large OR medium (fallback)
const candidates = vehicleType === VehicleType.TRUCK
  ? [SpotType.LARGE, SpotType.MEDIUM]
  : [required];

for (const spotType of candidates) {
  const spot = this.spots.find(s => !s.isOccupied && s.type === spotType);
  if (spot) return spot;
}
return null;
}
}

class ParkingLot {
  private tickets = new Map<string, Ticket>();
  private ticketToSpot = new Map<string, ParkingSpot>();

  constructor(
    private allocator: SpotAllocator,
    private feeStrategy: FeeStrategy
  ) {}

  park(vehicle: Vehicle): Ticket | null {
    const spot = this.allocator.findSpot(vehicle.type);
    if (!spot) return null;
    const ticket = spot.park(vehicle);
    this.tickets.set(ticket.id, ticket);
    this.ticketToSpot.set(ticket.id, spot);
    return ticket;
  }

  unpark(ticketId: string): { vehicle: Vehicle; fee: number } | null {
    const ticket = this.tickets.get(ticketId);
    const spot = this.ticketToSpot.get(ticketId);
    if (!ticket || !spot) return null;

    const vehicle = spot.unpark();
    if (!vehicle) return null;

    const fee = this.feeStrategy.calculate(ticket, new Date(), vehicle.type);
    this.tickets.delete(ticketId);
    this.ticketToSpot.delete(ticketId);
    return { vehicle, fee };
  }

  getAvailability(): Record<SpotType, number> {
    const counts = { [SpotType.SMALL]: 0, [SpotType.MEDIUM]: 0, [SpotType.LARGE]: 0 };
    for (const spot of this.allocator['spots']) {
      if (!spot.isOccupied) counts[spot.type]++;
    }
    return counts;
  }
}

```

```
}

// Usage
function buildParkingLot(): ParkingLot {
  const spots: ParkingSpot[] = [];
  for (let floor = 1; floor <= 3; floor++) {
    for (let i = 0; i < 10; i++) spots.push(new ConcreteSpot(`F${floor}-S${i}`, fl
    for (let i = 0; i < 20; i++) spots.push(new ConcreteSpot(`F${floor}-M${i}`, fl
    for (let i = 0; i < 5; i++) spots.push(new ConcreteSpot(`F${floor}-L${i}`, flo
  }
  return new ParkingLot(new SpotAllocator(spots), new HourlyFeeStrategy());
}
```

Edge Cases to Mention

- Lot full → return `null`, don't throw
- Invalid ticket on unpark → return `null`
- Truck fallback to medium spot
- Concurrent park on same spot → mention mutex/DB lock in production

4. Problem 2: In-Memory Rate Limiter

Requirements

- Limit requests per user/IP to N requests per window (e.g., 100 req/min)
- Sliding window algorithm (not fixed window)
- `isAllowed(key: string): boolean`
- In-memory; mention Redis for production

SOLID Mapping

Principle	Application
S	<code>SlidingWindowLimiter</code> only rate-limits; no HTTP concerns
O	Swap <code>SlidingWindowLimiter</code> for <code>TokenBucketLimiter</code> via interface
D	Controller depends on <code>RateLimiter</code> interface

Full Implementation – Sliding Window Log

```
interface RateLimiter {
  isAllowed(key: string): boolean;
  getRemaining(key: string): number;
}

class SlidingWindowRateLimiter implements RateLimiter {
  private logs = new Map<string, number[]>(); // key → timestamps

  constructor(
    private maxRequests: number,
    private windowMs: number
  ) {}

  isAllowed(key: string): boolean {
    const now = Date.now();
    const windowStart = now - this.windowMs;

    let timestamps = this.logs.get(key) ?? [];
    // Remove expired entries
    timestamps = timestamps.filter(t => t > windowStart);

    if (timestamps.length >= this.maxRequests) {
      this.logs.set(key, timestamps);
      return false;
    }

    timestamps.push(now);
    this.logs.set(key, timestamps);
    return true;
  }

  getRemaining(key: string): number {
    const now = Date.now();
    const windowStart = now - this.windowMs;
    const timestamps = (this.logs.get(key) ?? []).filter(t => t > windowStart);
    return Math.max(0, this.maxRequests - timestamps.length);
  }
}

// Sliding Window Counter (memory-efficient variant)
class SlidingWindowCounterLimiter implements RateLimiter {
  private windows = new Map<string, { current: Window; previous: Window }>();

  constructor(
    private maxRequests: number,
    private windowMs: number
  ) {}
```

```

isAllowed(key: string): boolean {
  const now = Date.now();
  const currentWindowId = Math.floor(now / this.windowMs);
  let entry = this.windows.get(key);

  if (!entry || entry.current.id !== currentWindowId) {
    const previous = entry?.current.id === currentWindowId - 1 ? entry.current :
    entry = { current: { id: currentWindowId, count: 0 }, previous };
    this.windows.set(key, entry);
  }

  const elapsed = now % this.windowMs;
  const weight = 1 - elapsed / this.windowMs;
  const estimated = entry.current.count + entry.previous.count * weight;

  if (estimated >= this.maxRequests) return false;
  entry.current.count++;
  return true;
}

getRemaining(key: string): number {
  return this.maxRequests; // simplified
}
}

interface Window { id: number; count: number; }

// Middleware usage
function rateLimitMiddleware(limiter: RateLimiter) {
  return (req: { ip: string }, res: { status: (n: number) => { json: (b: object) =
    if (!limiter.isAllowed(req.ip)) {
      return res.status(429).json({ error: 'Too many requests' });
    }
    next();
  };
}

```

Production Notes

- Redis: `ZADD key score member` + `ZREMRANGEBYSCORE` for sliding window log
- Distributed: all instances share Redis counter
- Add `Retry-After` header on 429

5. Problem 3: LRU Cache

Requirements

- `get(key)` and `put(key, value)` in $O(1)$
- Evict least recently used when capacity exceeded
- Generic over key/value types

SOLID Mapping

Principle	Application
S	Cache handles storage/eviction only
O	Pluggable eviction: LRU, LFU via <code>EvictionPolicy</code>
L	Any <code>EvictionPolicy</code> implementation interchangeable

Full Implementation

```
class LRUNode<K, V> {
  constructor(
    public key: K,
    public value: V,
    public prev: LRUNode<K, V> | null = null,
    public next: LRUNode<K, V> | null = null
  ) {}
}

class LRUCache<K, V> {
  private map = new Map<K, LRUNode<K, V>>();
  private head: LRUNode<K, V>; // dummy head (MRU side)
  private tail: LRUNode<K, V>; // dummy tail (LRU side)

  constructor(private capacity: number) {
    this.head = new LRUNode<K, V>(null as unknown as K, null as unknown as V);
    this.tail = new LRUNode<K, V>(null as unknown as K, null as unknown as V);
    this.head.next = this.tail;
    this.tail.prev = this.head;
  }

  get(key: K): V | undefined {
    const node = this.map.get(key);
    if (!node) return undefined;
    this.moveToFront(node);
    return node.value;
  }

  put(key: K, value: V): void {
    const existing = this.map.get(key);
    if (existing) {
      existing.value = value;
      this.moveToFront(existing);
      return;
    }

    const node = new LRUNode(key, value);
    this.map.set(key, node);
    this.addToFront(node);

    if (this.map.size > this.capacity) {
      const lru = this.removeTail();
      this.map.delete(lru.key);
    }
  }

  private moveToFront(node: LRUNode<K, V>): void {
    this.removeNode(node);
```

```
    this.addToFront(node);
  }

  private addToFront(node: LRUNode<K, V>): void {
    node.prev = this.head;
    node.next = this.head.next;
    this.head.next!.prev = node;
    this.head.next = node;
  }

  private removeNode(node: LRUNode<K, V>): void {
    node.prev!.next = node.next;
    node.next!.prev = node.prev;
  }

  private removeTail(): LRUNode<K, V> {
    const lru = this.tail.prev!;
    this.removeNode(lru);
    return lru;
  }
}
```

6. Problem 4: Snake & Ladder

Requirements

- N players, board size (e.g., 100)
- Snakes and ladders at fixed positions
- Dice roll 1–6, exact finish required to win
- Support multiple dice / custom rules (extensibility)

SOLID Mapping

Principle	Application
S	<code>Board</code> manages positions; <code>Game</code> manages turns; <code>Dice</code> rolls
O	Add power-ups via <code>MoveModifier</code> without changing <code>Game</code>
S	<code>Snake</code> and <code>Ladder</code> both implement <code>BoardEntity</code>

Full Implementation

```
interface BoardEntity {
  apply(position: number): number;
}

class Snake implements BoardEntity {
  constructor(private head: number, private tail: number) {}
  apply(position: number): number {
    return position === this.head ? this.tail : position;
  }
}

class Ladder implements BoardEntity {
  constructor(private bottom: number, private top: number) {}
  apply(position: number): number {
    return position === this.bottom ? this.top : position;
  }
}

class Dice {
  roll(): number {
    return Math.floor(Math.random() * 6) + 1;
  }
}

class Player {
  constructor(public id: string, public name: string, public position = 0) {}
}

enum GameStatus { IN_PROGRESS, WON }

class SnakeLadderGame {
  private players: Player[];
  private currentIndex = 0;
  private status = GameStatus.IN_PROGRESS;
  private entities = new Map<number, BoardEntity>();

  constructor(
    players: Player[],
    private boardSize: number,
    snakes: Array<{ head: number; tail: number }>,
    ladders: Array<{ bottom: number; top: number }>,
    private dice: Dice = new Dice()
  ) {
    this.players = players;
    snakes.forEach(s => this.entities.set(s.head, new Snake(s.head, s.tail)));
    ladders.forEach(l => this.entities.set(l.bottom, new Ladder(l.bottom, l.top)))
  }
}
```

```
playTurn(): { player: string; roll: number; from: number; to: number; won: boolean } {
  if (this.status === GameStatus.WON) throw new Error('Game over');

  const player = this.players[this.currentIndex];
  const roll = this.dice.roll();
  const from = player.position;

  if (from + roll > this.boardSize) {
    // Exact finish rule: stay put
    this.nextPlayer();
    return { player: player.name, roll, from, to: from, won: false };
  }

  let to = from + roll;
  const entity = this.entities.get(to);
  if (entity) to = entity.apply(to);
  player.position = to;

  const won = to === this.boardSize;
  if (won) this.status = GameStatus.WON;
  else this.nextPlayer();

  return { player: player.name, roll, from, to, won };
}

private nextPlayer(): void {
  this.currentIndex = (this.currentIndex + 1) % this.players.length;
}

getWinner(): Player | null {
  return this.status === GameStatus.WON
    ? this.players.find(p => p.position === this.boardSize) ?? null
    : null;
}
```

7. Problem 5: Splitwise Expense Splitting

Requirements

- Users, groups, expenses
- Split types: equal, exact amounts, percentage
- Track who owes whom
- Simplify debts (optional but impressive)

SOLID Mapping

Principle	Application
S	<code>EqualSplitStrategy</code> , <code>ExactSplitStrategy</code> each handle one split type
O	New split type = new strategy class
D	<code>ExpenseService</code> depends on <code>SplitStrategy</code> interface

Full Implementation

```
enum SplitType { EQUAL, EXACT, PERCENTAGE }

interface SplitStrategy {
  split(total: number, participants: string[], metadata?: Record<string, number>):
}

class EqualSplitStrategy implements SplitStrategy {
  split(total: number, participants: string[]): Map<string, number> {
    const share = total / participants.length;
    return new Map(participants.map(p => [p, share]));
  }
}

class ExactSplitStrategy implements SplitStrategy {
  split(total: number, participants: string[], metadata: Record<string, number> =
    const sum = Object.values(metadata).reduce((a, b) => a + b, 0);
    if (Math.abs(sum - total) > 0.01) throw new Error('Exact splits must sum to to
    return new Map(Object.entries(metadata));
  }
}

class PercentageSplitStrategy implements SplitStrategy {
  split(total: number, participants: string[], metadata: Record<string, number> =
    const result = new Map<string, number>();
    for (const p of participants) {
      const pct = metadata[p] ?? 0;
      result.set(p, (pct / 100) * total);
    }
    return result;
  }
}

class Expense {
  constructor(
    public id: string,
    public paidBy: string,
    public amount: number,
    public splits: Map<string, number> // userId → share owed
  ) {}
}

class BalanceSheet {
  private balances = new Map<string, Map<string, number>>(); // from → { to → amou

  addExpense(expense: Expense): void {
    for (const [userId, share] of expense.splits) {
      if (userId === expense.paidBy) continue;
      this.addDebt(userId, expense.paidBy, share);
    }
  }
}
```

```

    }
  }

  private addDebt(from: string, to: string, amount: number): void {
    if (!this.balances.has(from)) this.balances.set(from, new Map());
    const debts = this.balances.get(from)!;
    debts.set(to, (debts.get(to) ?? 0) + amount);
  }

  getBalances(userId: string): Map<string, number> {
    return this.balances.get(userId) ?? new Map();
  }

  // Greedy debt simplification
  simplify(): Array<{ from: string; to: string; amount: number }> {
    const net = new Map<string, number>();
    for (const [from, debts] of this.balances) {
      for (const [to, amount] of debts) {
        net.set(from, (net.get(from) ?? 0) - amount);
        net.set(to, (net.get(to) ?? 0) + amount);
      }
    }

    const creditors = [...net.entries()].filter(([, v]) => v > 0.01).sort((a, b) => a[1] - b[1]);
    const debtors = [...net.entries()].filter(([, v]) => v < -0.01).sort((a, b) => a[1] - b[1]);
    const result: Array<{ from: string; to: string; amount: number }> = [];

    let i = 0, j = 0;
    while (i < debtors.length && j < creditors.length) {
      const amount = Math.min(-debtors[i][1], creditors[j][1]);
      result.push({ from: debtors[i][0], to: creditors[j][0], amount });
      debtors[i][1] += amount;
      creditors[j][1] -= amount;
      if (Math.abs(debtors[i][1]) < 0.01) i++;
      if (creditors[j][1] < 0.01) j++;
    }
    return result;
  }
}

class ExpenseService {
  private strategies: Record<SplitType, SplitStrategy> = {
    [SplitType.EQUAL]: new EqualSplitStrategy(),
    [SplitType.EXACT]: new ExactSplitStrategy(),
    [SplitType.PERCENTAGE]: new PercentageSplitStrategy(),
  };

  constructor(private balanceSheet: BalanceSheet) {}

  addExpense(
    paidBy: string,
    amount: number,

```

```
participants: string[],
splitType: SplitType,
metadata?: Record<string, number>
): Expense {
  const splits = this.strategies[splitType].split(amount, participants, metadata)
  const expense = new Expense(`EXP-${Date.now()}`, paidBy, amount, splits);
  this.balanceSheet.addExpense(expense);
  return expense;
}
```

8. Problem 6: BookMyShow Seat Booking

Requirements

- Theatre with rows and seats
- Show seat map with available/booked/locked states
- Lock seats for 5 minutes during checkout
- Confirm booking or release lock on timeout
- Prevent double booking

SOLID Mapping

Principle	Application
S	<code>SeatLockManager</code> handles locks; <code>BookingService</code> handles bookings
O	Add premium pricing via <code>PricingStrategy</code>
S	Single responsibility per state transition

Full Implementation

```
enum SeatStatus { AVAILABLE, LOCKED, BOOKED }

interface Seat {
  id: string;
  row: string;
  number: number;
  status: SeatStatus;
  lockedBy: string | null;
  lockExpiry: Date | null;
}

class SeatLockManager {
  private lockDurationMs: number;

  constructor(lockDurationMinutes = 5) {
    this.lockDurationMs = lockDurationMinutes * 60 * 1000;
  }

  lockSeats(seats: Seat[], userId: string): boolean {
    // Check all available first (atomic intent)
    if (seats.some(s => s.status !== SeatStatus.AVAILABLE)) return false;

    const expiry = new Date(Date.now() + this.lockDurationMs);
    for (const seat of seats) {
      seat.status = SeatStatus.LOCKED;
      seat.lockedBy = userId;
      seat.lockExpiry = expiry;
    }
    return true;
  }

  releaseExpiredLocks(seats: Seat[]): void {
    const now = new Date();
    for (const seat of seats) {
      if (
        seat.status === SeatStatus.LOCKED &&
        seat.lockExpiry &&
        seat.lockExpiry < now
      ) {
        seat.status = SeatStatus.AVAILABLE;
        seat.lockedBy = null;
        seat.lockExpiry = null;
      }
    }
  }

  confirmBooking(seats: Seat[], userId: string): boolean {
    for (const seat of seats) {
```

```

        if (seat.status !== SeatStatus.LOCKED || seat.lockedBy !== userId) return false;
        seat.status = SeatStatus.BOOKED;
        seat.lockedBy = null;
        seat.lockExpiry = null;
    }
    return true;
}

releaseLocks(seats: Seat[], userId: string): void {
    for (const seat of seats) {
        if (seat.status === SeatStatus.LOCKED && seat.lockedBy === userId) {
            seat.status = SeatStatus.AVAILABLE;
            seat.lockedBy = null;
            seat.lockExpiry = null;
        }
    }
}

class Theatre {
    seats: Seat[];

    constructor(rows: number, seatsPerRow: number) {
        this.seats = [];
        for (let r = 0; r < rows; r++) {
            const rowLabel = String.fromCharCode(65 + r);
            for (let n = 1; n <= seatsPerRow; n++) {
                this.seats.push({
                    id: `${rowLabel}${n}`,
                    row: rowLabel,
                    number: n,
                    status: SeatStatus.AVAILABLE,
                    lockedBy: null,
                    lockExpiry: null,
                });
            }
        }
    }

    getSeatMap(): Seat[] {
        return this.seats.map(s => ({ ...s }));
    }
}

class BookingService {
    constructor(
        private theatre: Theatre,
        private lockManager: SeatLockManager
    ) {}

    lockSeats(seatIds: string[], userId: string): boolean {
        this.lockManager.releaseExpiredLocks(this.theatre.seats);
    }
}

```

```
const seats = seatIds.map(id => this.theatre.seats.find(s => s.id === id!));
if (seats.some(s => !s)) return false;
return this.lockManager.lockSeats(seats, userId);
}

confirmBooking(seatIds: string[], userId: string): boolean {
  const seats = seatIds.map(id => this.theatre.seats.find(s => s.id === id!));
  return this.lockManager.confirmBooking(seats, userId);
}

cancelLock(seatIds: string[], userId: string): void {
  const seats = seatIds.map(id => this.theatre.seats.find(s => s.id === id!));
  this.lockManager.releaseLocks(seats, userId);
}
}
```

Production Notes

- Use Redis `SET key NX EX 300` for distributed seat locks
- DB: `UPDATE seats SET status='LOCKED' WHERE id IN (...) AND status='AVAILABLE'` (optimistic)
- WebSocket to push seat map updates to other users

9. Problem 7: Elevator System

Requirements

- Building with N floors and M elevators
- Request: internal (from inside) and external (from floor button)
- Direction: UP, DOWN, IDLE
- Optimize: nearest elevator, SCAN algorithm

SOLID Mapping

Principle	Application
S	Scheduler assigns requests; Elevator moves; Controller orchestrates
O	Swap NearestElevatorStrategy for LoadBalancedStrategy
D	Controller depends on ElevatorScheduler interface

Full Implementation

```
enum Direction { UP, DOWN, IDLE }
enum RequestType { INTERNAL, EXTERNAL }

interface ElevatorRequest {
  floor: number;
  direction: Direction;
  type: RequestType;
}

class Elevator {
  id: string;
  currentFloor = 0;
  direction = Direction.IDLE;
  private requests = new Set<number>();

  constructor(id: string) {
    this.id = id;
  }

  addRequest(floor: number): void {
    this.requests.add(floor);
    this.updateDirection();
  }

  private updateDirection(): void {
    if (this.requests.size === 0) {
      this.direction = Direction.IDLE;
      return;
    }
    const targets = [...this.requests].sort((a, b) => a - b);
    if (this.currentFloor < targets[targets.length - 1]) {
      this.direction = Direction.UP;
    } else if (this.currentFloor > targets[0]) {
      this.direction = Direction.DOWN;
    }
  }

  step(): boolean {
    if (this.requests.size === 0) return false;

    if (this.direction === Direction.UP) this.currentFloor++;
    else if (this.direction === Direction.DOWN) this.currentFloor--;

    if (this.requests.has(this.currentFloor)) {
      this.requests.delete(this.currentFloor);
    }

    this.updateDirection();
  }
}
```

```
        return true;
    }

    canServe(request: ElevatorRequest): boolean {
        if (this.direction === Direction.IDLE) return true;
        if (request.direction === Direction.UP && this.direction === Direction.UP) {
            return request.floor >= this.currentFloor;
        }
        if (request.direction === Direction.DOWN && this.direction === Direction.DOWN) {
            return request.floor <= this.currentFloor;
        }
        return false;
    }

    distanceTo(floor: number): number {
        return Math.abs(this.currentFloor - floor);
    }
}

interface ElevatorScheduler {
    assign(request: ElevatorRequest, elevators: Elevator[]): Elevator;
}

class NearestElevatorScheduler implements ElevatorScheduler {
    assign(request: ElevatorRequest, elevators: Elevator[]): Elevator {
        const serving = elevators.filter(e => e.canServe(request));
        const pool = serving.length > 0 ? serving : elevators;
        return pool.reduce((best, e) =>
            e.distanceTo(request.floor) < best.distanceTo(request.floor) ? e : best
        );
    }
}

class ElevatorController {
    constructor(
        private elevators: Elevator[],
        private scheduler: ElevatorScheduler
    ) {}

    requestElevator(floor: number, direction: Direction): Elevator {
        const request: ElevatorRequest = { floor, direction, type: RequestType.EXTERNAL };
        const elevator = this.scheduler.assign(request, this.elevators);
        elevator.addRequest(floor);
        return elevator;
    }

    selectFloor(elevatorId: string, floor: number): void {
        const elevator = this.elevators.find(e => e.id === elevatorId!);
        elevator.addRequest(floor);
    }

    runStep(): void {
```

```
    for (const elevator of this.elevators) {  
      elevator.step();  
    }  
  }  
}
```

10. Problem 8: Vending Machine

Requirements

- States: Idle, HasMoney, Dispensing, OutOfStock
- Accept coins/notes, select product, dispense change
- State pattern for transitions
- Inventory management

SOLID Mapping

Principle	Application
S	Each state class handles only its transitions
O	New state = new class, no switch/if chains in machine
D	<code>VendingMachine</code> delegates to <code>VendingState</code> interface

Full Implementation

```
interface Product {
  id: string;
  name: string;
  price: number;
  quantity: number;
}

interface VendingState {
  insertMoney(machine: VendingMachine, amount: number): void;
  selectProduct(machine: VendingMachine, productId: string): void;
  dispense(machine: VendingMachine): void;
  cancel(machine: VendingMachine): void;
}

class VendingMachine {
  state: VendingState;
  balance = 0;
  selectedProduct: Product | null = null;
  products = new Map<string, Product>();

  constructor() {
    this.state = new IdleState();
  }

  setState(state: VendingState): void {
    this.state = state;
  }

  insertMoney(amount: number): void { this.state.insertMoney(this, amount); }
  selectProduct(id: string): void { this.state.selectProduct(this, id); }
  dispense(): void { this.state.dispense(this); }
  cancel(): void { this.state.cancel(this); }
}

class IdleState implements VendingState {
  insertMoney(machine: VendingMachine, amount: number): void {
    machine.balance += amount;
    machine.setState(new HasMoneyState());
  }
  selectProduct(): void { console.log('Insert money first'); }
  dispense(): void { console.log('Insert money first'); }
  cancel(): void { console.log('Nothing to cancel'); }
}

class HasMoneyState implements VendingState {
  insertMoney(machine: VendingMachine, amount: number): void {
    machine.balance += amount;
  }
}
```

```
selectProduct(machine: VendingMachine, productId: string): void {
  const product = machine.products.get(productId);
  if (!product || product.quantity === 0) {
    console.log('Product unavailable');
    return;
  }
  if (machine.balance < product.price) {
    console.log('Insufficient balance');
    return;
  }
  machine.selectedProduct = product;
  machine.setState(new DispensingState());
  machine.dispense();
}

dispense(): void { console.log('Select a product first'); }

cancel(machine: VendingMachine): void {
  console.log(`Refunding ${machine.balance}`);
  machine.balance = 0;
  machine.setState(new IdleState());
}
}

class DispensingState implements VendingState {
  insertMoney(): void { console.log('Dispensing in progress'); }
  selectProduct(): void { console.log('Dispensing in progress'); }

  dispense(machine: VendingMachine): void {
    const product = machine.selectedProduct!;
    product.quantity--;
    const change = machine.balance - product.price;
    console.log(`Dispensed: ${product.name}, Change: ${change}`);
    machine.balance = 0;
    machine.selectedProduct = null;
    machine.setState(new IdleState());
  }

  cancel(): void { console.log('Cannot cancel during dispense'); }
}
```

11. Evaluation Criteria

Scoring Rubric (What Interviewers Actually Use)

Dimension	Weight	Strong Signal	Weak Signal
Requirements	15%	Asks 5+ clarifying questions, defines scope	Starts coding immediately
Design	25%	Clean class diagram, appropriate patterns	God class, no interfaces
Code Quality	25%	Readable, typed, no bugs on happy path	Syntax errors, wrong logic
Extensibility	15%	Strategy/Factory for variation	Hard-coded if/else chains
Edge Cases	10%	Handles null, full, duplicate	Only happy path
Communication	10%	Explains trade-offs, thinks aloud	Silent coding

Red Flags (Instant Downgrade)

- Cannot run or walk through code
- Everything in one 200-line function
- Ignores interviewer hints
- No questions about requirements
- Over-engineering with 15 classes for simple problem

Green Flags (L5 / 80 LPA Hire)

- "I'll define the interface first, then implement"
- "For production I'd use Redis here because..."
- Writes code incrementally and tests mentally
- Acknowledges trade-off: "HashMap gives O(1) but uses more memory"
- Handles "what if we add X?" without rewriting

Senior vs Mid-Level Expectations

Aspect	Mid-Level (30 LPA)	Senior (80 LPA)
Patterns	Knows names	Applies correctly, not excessively
Concurrency	Ignores	Mentions locks, race conditions
Testing	Skips	Writes or describes test cases
Scale	Single machine	Discusses distributed variant
Time mgmt	Runs out of time	Delivers core path + extensibility

12. 10 Machine Coding Interview Q&A

Q1: How do you manage time in a 90-minute machine coding round?

Answer: I split time rigidly: 10 minutes for requirements and scope, 10 minutes for API/class design, 60 minutes for coding the core path first (happy path always), and 10 minutes for edge cases and trade-off discussion. I never spend more than 15 minutes on design — interviewers want to see code. If I'm running behind at the 45-minute mark, I skip nice-to-haves and verbalize what I'd add later. The core path (e.g., park and unpark for parking lot) must work; secondary features (fee strategies, multi-floor UI) come second.

Q2: Should I use TypeScript, Java, or Python?

Answer: Use whatever the company specifies or what you listed on your resume. For Indian 80 LPA rounds, Java and TypeScript are most common. TypeScript shows frontend/full-stack roles; Java shows backend/OOD strength. Python is acceptable but less common for OOD rounds. Consistency matters more than language — interviewers evaluate design, not syntax. If unspecified, ask: "Is TypeScript OK?" and proceed.

Q3: How much design do I need before coding?

Answer: Define enums, 3–5 core classes, and key interfaces on paper or in comments — roughly 10 minutes. A full UML diagram is overkill; a box diagram with arrows between `ParkingLot`,

`SpotAllocator`, and `FeeStrategy` is sufficient. The interviewer wants to see you identify entities (nouns from requirements) and relationships (has-a, uses-a). Start coding as soon as the API surface is clear.

Q4: What if I can't finish the full implementation?

Answer: Prioritize a working core path over incomplete features. A working `park()` and `unpark()` with one vehicle type beats a half-implemented multi-floor system. Explicitly tell the interviewer: "I'll implement the happy path first, then extend for trucks." In the last 10 minutes, walk through unimplemented parts verbally with pseudocode. Interviewers at L5 level evaluate prioritization as much as completion.

Q5: How do I handle concurrency questions in machine coding?

Answer: For in-memory rounds, implement single-threaded logic correctly, then discuss concurrency separately: "In production, I'd use `synchronized` or a `ReentrantLock` around `park()` to prevent double-assignment." For BookMyShow, mention optimistic locking: `UPDATE seats SET status='LOCKED' WHERE id=? AND status='AVAILABLE'`. For rate limiters, mention Redis atomic operations. You don't need to implement locks unless explicitly asked.

Q6: When should I use design patterns vs keep it simple?

Answer: Apply patterns when they solve a stated extensibility requirement, not preemptively. Use Strategy when the prompt says "different pricing models" or "multiple split types." Use State for vending machine or order lifecycle. Use Factory when object creation is complex. Avoid pattern name-dropping without code — interviewers prefer a simple correct solution over FactoryAbstractSingleton chaos. Rule of thumb: if you'd need a switch on type in 3+ places, extract a strategy.

Q7: How important is running code in the interview?

Answer: Very important at companies like Flipkart and Razorpay where you code in their IDE. At Google/Meta, whiteboard or shared doc is common — "running" means walking through with sample input. Always mentally trace: "If I call `park(car)`, spot M1 gets occupied and ticket T1 is returned." If the platform allows, run 2–3 test cases. Syntax errors are forgiven; logic errors are not.

Q8: How do I extend my solution when the interviewer adds a requirement?

Answer: This is intentional — they test Open/Closed Principle. If asked "add electric vehicle charging spots," add `SpotType.CHARGING` and `ChargingFeeStrategy` without modifying `SpotAllocator` core logic. Say: "I'll extend the enum and add a new strategy class." Never rewrite existing working code; extend via new classes. This is exactly what L5 interviewers want to see.

Q9: What edge cases should I always handle?

Answer: Universal edge cases: null/undefined input, empty collections, duplicate operations (double park same vehicle), resource exhaustion (lot full, cache at capacity), invalid state transitions (unpark with invalid ticket), and boundary values (0 floors, 1-item cache). Mention them even if you don't code all: "I should also handle the case where unpark is called twice with the same ticket."

Q10: How is machine coding different from LeetCode DSA?

Answer: DSA tests algorithmic thinking on isolated problems (shortest path, DP). Machine coding tests system building: multiple classes, state management, business rules, and extensibility. DSA answers are optimal when Big-O is correct; machine coding answers are optimal when design is clean and code is maintainable. At 80 LPA, you need both rounds — DSA for screening, machine coding for team matching. Machine coding rewards engineering judgment over mathematical insight.

Quick Reference Checklist

Before interview:

- ☐ Practice 8 classic problems timed at 90 min
- ☐ Memorize SOLID one-liner examples
- ☐ Prepare 5 clarifying questions per problem type

During interview:

- ☐ Repeat requirements back to interviewer
- ☐ Write enums and interfaces first
- ☐ Code happy path before edge cases
- ☐ Think aloud continuously
- ☐ Mention production scaling in last 5 min

After coding:

- ☐ Walk through 2 test cases
- ☐ Answer "what would you improve?"
- ☐ Discuss one trade-off you made

13. Frontend machine coding (1 CR)

Atlassian, Uber, Razorpay, and some Google FE loops ask a **working React widget**, not Splitwise.

Full list: [20-1cr-product-company-complete.pdf](#) \$6.

90-minute FE framework

Min	Do
0-10	Data shape, keyboard, empty/error, mobile?
10-20	Component tree + TypeScript types + state owner
20-70	Happy path + one interaction
70-85	Debounce, loading, empty, basic a11y
85-90	Virtualize / tests / what you'd do next

Must-build (timed, React + TS)

Widget	Must work
Autocomplete	Debounce 200ms, abort in-flight, arrow keys, highlight
Infinite scroll	IntersectionObserver, no duplicate pages
Tic-tac-toe	Win detect, reset, no stale state
Kanban column	Reorder cards in state
Nested comments	Reply + collapse

Put them in `mern-mastery/frontend-mc/`. Do **autocomplete** on Day 139 and a second widget in Phase 7G.

Autocomplete – what they want to hear

```
function useDebounced<T>(value: T, ms: number) {
  const [v, setV] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setV(value), ms);
    return () => clearTimeout(t);
  }, [value, ms]);
  return v;
}
// Fetch: new AbortController() per keystroke; abort previous.
// ally: role="listbox", aria-activedescendant, input aria-autocomplete="list"
```

Fail signals: no debounce (spam API), no loading/empty, `index` as React key, cannot use keyboard.

Target: Google L4–L5, Meta E4–E5, Amazon SDE2–3, Atlassian, Uber — ~1 CR product loops (backend MC + frontend MC).

Machine Coding Interview – 50 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

This bank covers machine coding rounds with classic problems like Splitwise, BookMyShow, and Tic Tac Toe, including design patterns and implementation strategy for product company interviews. Spell out abbreviations on first use.

1. What is a machine coding round?

A machine coding round is a hands-on interview where you build a working program from scratch in 90–120 minutes—typically in Java, Python, or JavaScript. Unlike Low Level Design (LLD), you write runnable code, not just class diagrams. Problems simulate real features: expense splitting, ticket booking, or game logic. Interviewers evaluate code quality, design, and completeness under time pressure.

2. What do interviewers evaluate in machine coding?

Interviewers assess: (1) working code that handles core flows, (2) clean Object-Oriented Programming (OOP) design with appropriate classes, (3) extensibility (easy to add features), (4) edge case handling, (5) naming and readability, (6) optional bonus: tests, threading, or design patterns. They care less about perfect UI and more about solid backend logic and structure.

3. What is the recommended approach for machine coding?

Follow this timeline: 0–10 min: clarify requirements and list features (must-have vs nice-to-have). 10–20 min: identify entities, draw class diagram, define interfaces. 20–70 min: implement core happy path first, then edge cases. 70–85 min: demo walkthrough, add one extension if time permits. 85–90 min: refactor naming, remove duplication. Always build incrementally—working code early beats perfect design late.

4. How do you design Splitwise in machine coding?

Splitwise tracks shared expenses among friends and simplifies debts. Core entities: `User`, `Expense`, `Split` (equal, exact, or percentage), `Group`, `BalanceSheet`. Operations: add expense with splits, view balances between users, simplify debts (minimize transactions). Use Strategy

pattern for split types. Store balances in a map: $(\text{userA}, \text{userB}) \rightarrow \text{amount}$. No database needed—in-memory maps suffice.

5. What classes do you need for Splitwise?

Minimum classes: `User` (id, name), `Expense` (id, amount, paidBy, splits, splitType), `Split` (user, amountOrShare), `ExpenseService` (addExpense, getBalances), `BalanceSheet` (map of debts), `SplitStrategy` interface with `EqualSplit`, `ExactSplit`, `PercentageSplit` implementations. Optional: `Group` (members, group expenses), `DebtSimplifier` (reduce transactions). Keep entities plain; put logic in services.

6. How does expense splitting work in Splitwise?

When user A pays 300 split equally among A, B, C: each owes 100. Balance sheet: B owes A 100, C owes A 100. For exact split: A pays 300, B's share 100, C's share 200—B owes A 100, C owes A 200. For percentage: C pays 300, A=50%, B=30%, C=20%—A owes C 150, B owes C 90 (C paid so C is creditor). Update balance map on each expense addition.

7. What is debt simplification in Splitwise?

Debt simplification reduces circular debts to minimum transactions. If A owes B 100 and B owes C 100 and C owes A 100, net balance is zero—no payments needed. Algorithm: compute net balance per user (total owed minus total owing), then match creditors with debtors greedily. Use priority queue or two-pointer on sorted net balances. Optional feature—implement after core expense tracking works.

8. How do you design BookMyShow in machine coding?

BookMyShow books movie tickets for a theater. Entities: `Movie`, `Theater`, `Screen`, `Show` (movie + screen + time), `Seat` (row, number, type, status), `Booking`. Flow: browse shows → select seats → lock seats → pay → confirm booking. `SeatLockService` holds seats for 5–10 minutes during payment. Use Enum for seat status: AVAILABLE, LOCKED, BOOKED.

9. What is seat locking in BookMyShow?

Seat locking temporarily reserves selected seats while the user completes payment—prevents double booking. On seat selection, mark seats LOCKED with expiry timestamp and user session ID.

Background thread or scheduled task releases expired locks. On payment success, mark BOOKED; on timeout/failure, revert to AVAILABLE. Critical for concurrent booking scenarios in interviews.

10. How do you handle concurrent seat booking?

Use synchronized blocks or `ReentrantLock` per show (not global lock) when checking and locking seats. Pattern: lock show → verify all seats AVAILABLE → mark LOCKED → release lock. Database version: `UPDATE seats SET status='LOCKED' WHERE id IN (...) AND status='AVAILABLE'` — optimistic concurrency. Mention both in-memory (interview) and database (production) approaches.

11. What classes do you need for BookMyShow?

`Movie` (id, title, duration), `Theater` (id, name, screens), `Screen` (id, seats layout), `Seat` (row, number, type, status), `Show` (id, movie, screen, startTime, pricing), `Booking` (id, show, seats, user, status), `BookingService`, `SeatLockService`, `PaymentService` (mock). Factory for seat layout generation (rows × seats per row). Strategy for pricing (regular, premium, weekend).

12. How do you design Tic Tac Toe in machine coding?

Entities: `Board` (N×N grid), `Player` (name, symbol X/O), `Game` (board, two players, current turn, status). Operations: `makeMove(row, col)`, `getStatus()`, `printBoard()`. `Board` validates moves and checks win/draw. `Game` orchestrates turn alternation. Start with 3×3; extend to N×N. Win check: row, column, diagonal after each move—O(N) per move.

13. How do you extend Tic Tac Toe to N×N board?

Generalize win condition: K consecutive marks in a row (usually K=N). Board becomes N×N array. Win check after each move: check row, column, two diagonals passing through last move—O(N) instead of checking entire board. For 4×4 with K=3, check 3-in-a-row. Parameterize board size and win length in constructor.

14. How do you design multi-player Tic Tac Toe?

Extend from 2 players to N players with symbols X, O, A, B, etc. `Game` holds list of players and current player index. After each move, advance `(currentIndex + 1) % numPlayers`. Win check

same but for any player's symbol. Handle up to 26 players (letters) or use numbered symbols. Same Board class works—only Game orchestration changes.

15. What design patterns apply to Tic Tac Toe?

Strategy pattern for computer player AI: `EasyAI` (random move), `MediumAI` (block opponent wins), `HardAI` (minimax algorithm). Factory for creating games (3×3, 4×4, multi-player).

Observer for notifying UI of board changes (if building with display). State pattern if adding game phases (SETUP, PLAYING, FINISHED). Mention patterns but do not over-engineer for a 90-minute round.

16. How do you add a computer player to Tic Tac Toe?

Implement `AIPlayer` interface with `getMove(board)`. Easy: collect empty cells, pick random.

Medium: if AI can win in one move, take it; if opponent can win, block; else random. Hard: minimax with alpha-beta pruning—explore all game states, assume optimal play. Minimax is impressive but time-consuming; implement Easy/Medium first, describe Hard if asked.

17. What is the Parking Lot machine coding problem?

Design a parking lot with multiple floors, spot types (compact, large, handicapped), and vehicle types (car, motorcycle, truck). Operations: park vehicle (assign spot, issue ticket), unpark (calculate fee, free spot). Classes: `ParkingLot`, `Floor`, `Spot`, `Vehicle`, `Ticket`, `ParkingFeeStrategy`. Similar to LLD but you must write working code with a main method demo.

18. What is the Snake and Ladder machine coding problem?

Board with 100 cells, snakes (slide down), and ladders (climb up). Multiple players roll dice, move forward, apply snake/ladder if landing on one. First to cell 100 wins. Classes: `Board` (cells, snakes, ladders maps), `Player`, `Game` (turn management, dice roll). Handle exact 100 rule: if roll overshoots 100, stay put. Simple but tests basic OOP and game loop.

19. What is the Elevator machine coding problem?

Multiple elevators serve multiple floors. Users request up/down from floor; users inside request destination. Elevator moves, picks up, drops off. Algorithm: SCAN or LOOK for efficient movement.

Classes: `Elevator` (current floor, direction, passengers), `ElevatorController` (dispatch requests). State pattern for elevator direction. Discuss thread safety if multiple requests concurrent.

20. How do you structure code for machine coding?

Use package/folder structure: `models/` (entities), `services/` (business logic), `enums/` (status types), `Main.java` (demo). One class per file. Service classes receive dependencies via constructor (dependency injection). Avoid static mutable state. Separate concerns: Board does not print; Game or a Display class handles output. Clean structure impresses interviewers more than clever one-liners.

21. Should you use a database in machine coding?

No—use in-memory data structures (HashMap, ArrayList) unless interviewer explicitly asks for persistence. Mention: "In production I'd use PostgreSQL with these tables..." but implement with maps for speed. Focus time on business logic, not Object-Relational Mapping (ORM) setup. A `Repository` interface with `InMemoryRepository` implementation shows good design without database overhead.

22. How do you handle input in machine coding?

Two approaches: (1) hardcoded demo in main method showing key flows—fastest and most common, (2) simple console menu reading user input. Prefer hardcoded demo with 2–3 scenarios (happy path + edge case) unless interviewer asks for interactive input. Print clear output showing state changes: "User A paid 300. Balances: B owes A 150."

23. What edge cases matter in Splitwise?

Zero amount expense (reject), single person expense (no splits needed), split amounts not summing to total (validate), negative amounts (reject), user not in group, duplicate expense IDs, empty group, percentage splits not totaling 100%, and simplifying when no debts exist. Validate inputs in `addExpense` before updating balances.

24. What edge cases matter in BookMyShow?

Booking already booked seats (reject), booking locked seats by another user (reject), lock expiry during payment (handle gracefully), empty seat selection (reject), show already started (optional

restriction), concurrent lock on same seat (synchronized), and cancel booking (revert seats to AVAILABLE). Test with two users trying to book the same seat simultaneously.

25. What edge cases matter in Tic Tac Toe?

Move on occupied cell (reject), move after game over (reject), move out of bounds (reject), draw detection (board full, no winner), winning on last move (check before declaring draw), and single player mode. Handle invalid input gracefully with error messages, not exceptions crashing the program.

26. How do you implement undo in machine coding?

Command pattern: each action (move, expense, booking) is a Command object with `execute()` and `undo()`. Maintain a stack of executed commands. Undo pops and calls `undo()`. Example: Tic Tac Toe move command stores previous board state. Splitwise undo reverses balance changes. Mention pattern; implement only if time permits or interviewer asks.

27. What is the difference between machine coding and Low Level Design (LLD)?

LLD focuses on class diagrams, design patterns, and explaining relationships—often without running code. Machine coding requires working, executable code demonstrating core functionality. LLD allows more discussion time; machine coding demands faster implementation. Both test OOP design, but machine coding additionally tests coding speed and pragmatism under pressure.

28. How much time for each phase in a 90-minute machine coding round?

Requirements clarification: 5–10 min. Design (classes, interfaces): 10–15 min. Core implementation: 40–50 min. Edge cases and bug fixes: 10–15 min. Demo and discussion: 10–15 min. If running behind at 45 min, cut nice-to-have features and ensure happy path works. A working basic solution beats an incomplete advanced one.

29. Should you write tests in machine coding?

Write tests only if time remains after core implementation—JUnit or simple assert statements. One or two tests for critical logic (split calculation, win detection) show good habits. Do not spend 20 minutes on test infrastructure. Mention: "I'd add comprehensive unit tests with mocked dependencies in a real project." Prioritize working demo over test coverage.

30. What language should you use for machine coding?

Use the language you are fastest in and the company expects—Java is most common in Indian product companies (Flipkart, Amazon, PhonePe). Python is acceptable if allowed and you're proficient. JavaScript/TypeScript for frontend-focused roles. Stick to one language; do not switch mid-interview. Use standard library collections—avoid obscure frameworks.

31. How do you design a LRU Cache in machine coding?

Least Recently Used (LRU) Cache: `get(key)` and `put(key, value)` in $O(1)$ with fixed capacity. Use HashMap + Doubly Linked List. Map stores key→node; list maintains access order (head=most recent). On get: move node to head. On put: add to head; if over capacity, remove tail. Implement in 20–30 minutes—common Amazon/Flipkart problem.

32. How do you design a Rate Limiter in machine coding?

Implement Token Bucket or Fixed Window rate limiter. `allowRequest(userId)` returns true/false. Token Bucket: refill tokens at fixed rate, consume one per request. Store user state in HashMap. Include `Clock` interface for testability. Handle multiple users independently. Demonstrate with rapid calls showing rejection after limit exceeded.

33. How do you design in-memory Redis in machine coding?

Support commands: `SET key value`, `GET key`, `DEL key`, `EXPIRE key seconds`, optionally `LPUSH/RPOP` for lists. Use HashMap for storage; separate HashMap for expiry timestamps. Background thread or check-on-access for expired keys. Demonstrates data structure knowledge and API design. Simpler variant: key-value store without expiry.

34. What is the Vending Machine machine coding problem?

Vending machine dispenses products on coin insertion. States: IDLE, ACCEPTING_MONEY, DISPENSING. Accept coins/notes, select product, check inventory and payment, dispense, return change. Classes: `VendingMachine`, `Product`, `Inventory`, `Coin` enum. State pattern for machine states. Handle insufficient payment, out of stock, and exact change scenarios.

35. How do you design a Chess game in machine coding?

Too complex for 90 minutes—scope down to: board setup, piece movement validation for 2–3 piece types (pawn, rook, king), turn management, check detection. Factory for piece creation. Strategy or polymorphism for piece movement rules. Do not implement full chess—interviewer expects scoping discussion. 3×3 or 5×5 variant is more realistic.

36. How do you design a Library Management System?

Entities: `Book`, `BookCopy` (barcode), `Member`, `Loan`, `Reservation`. Operations: search catalog, checkout (if available), return (calculate fine), reserve. `CatalogService`, `LoanService`. Fine calculation via Strategy (daily rate). Due date tracking with simple date comparison. In-memory catalog with HashMap indexed by ISBN and title.

37. What is immutability in machine coding?

Make value objects (Money, Split, Coordinate) immutable—all fields final, no setters, defensive copies. Prevents accidental state corruption during game logic or balance calculations. Example: `Split` with final user and amount. Immutable objects are thread-safe by default. Shows mature design thinking in a timed round.

38. How do you show extensibility in machine coding?

Use interfaces for volatile behavior: `SplitStrategy`, `PricingStrategy`, `AIPlayer`. Add new split type by creating new class implementing interface—no changes to existing code (Open/Closed Principle). Mention future extensions verbally: "To add CREDIT_CARD payment, I'd implement PaymentMethod interface." One extension example in code is sufficient.

39. How do you handle errors in machine coding?

Define custom exceptions: `SeatAlreadyBookedException`, `InvalidSplitException`. Validate inputs at service entry points. Return clear error messages, not stack traces. Use `Optional` for not-found cases. Do not over-engineer—2–3 custom exceptions for domain errors plus `IllegalArgumentException` for bad inputs is enough.

40. What is a singleton and when to use in machine coding?

Singleton ensures one instance of a class (e.g., `ParkingLot`, `GameManager`). Use only when truly one instance needed. Implement with private constructor and static `getInstance()`, or prefer passing instance explicitly. Interviewers often discourage Singleton—mention it but prefer creating one instance in main and passing via constructors.

41. How do you demo your machine coding solution?

Walk through 2–3 scenarios in main method with printed output: "Scenario 1: Equal split among 3 users. Scenario 2: Exact split with validation error. Scenario 3: Book seats, show lock expiry." Explain code structure briefly while running. Show you tested edge cases. Confident demo matters as much as code quality.

42. How do you design a Meeting Scheduler?

Given multiple people's calendars (busy intervals), find a meeting slot of duration D that works for all. Input: list of busy intervals per person. Merge intervals per person, find free gaps across all people. Classes: `Person`, `Interval`, `MeetingScheduler`. Sort intervals, sweep through time. LeetCode 253 variant. Good for interval problem practice.

43. How do you design a Task Management System (like Jira)?

Entities: `Task` (id, title, status, assignee, priority), `User`, `Board` (columns: TODO, IN_PROGRESS, DONE). Operations: create task, assign, change status, filter by assignee/priority. State pattern for status transitions (validate TODO→IN_PROGRESS, not TODO→DONE directly). Observer for status change notifications. Scope to CRUD + status workflow.

44. What threading concerns appear in machine coding?

BookMyShow seat locking, rate limiter token refill, cache expiry cleanup. Use `synchronized` or `ReentrantLock` for critical sections. Prefer lock per resource (per show) over global lock. Mention thread safety even if you do not implement full threading—"I'd synchronize the `lockSeats` method to handle concurrent requests." Shows awareness.

45. How do you optimize for interview time pressure?

Cut scope aggressively: 3×3 before $N \times N$, equal split before percentage, single theater before multi-theater. Hardcode data instead of building input parsers. Skip logging framework—use `System.out.println`. Implement must-haves first; mention nice-to-haves verbally. Working happy path at 60 minutes leaves 30 for polish and edge cases.

46. What common mistakes fail machine coding candidates?

No working code at end, God classes with all logic in main, no input validation, ignoring edge cases, starting to code without design, over-engineering with too many patterns, spending too long on one feature, not communicating thought process, and mutable shared state causing bugs. Practice 2–3 full problems timed before the interview.

47. Summarize machine coding interview checklist?

Clarify scope → design classes (5 min) → implement happy path → handle edge cases → demo with printed scenarios → discuss extensibility. Use clean OOP, Strategy/Factory where natural, in-memory storage, and hardcoded demos. Communicate throughout. Prioritize working code over perfect architecture. Practice Splitwise, BookMyShow, and Tic Tac Toe until you can complete each in 75 minutes.

48. How do you design a URL shortener in machine coding?

`UrlShortenerService` with `shorten(longUrl)` and `resolve(shortCode)`. Base-62 encode auto-increment ID or random 6-char code with collision check. Store in `HashMap`. Validate URLs, handle not-found. Optional: click counter, expiration. Simpler than system design version—no database, no distributed ID. Complete in 30–40 minutes.

49. How do you design a parking lot in machine coding?

Multiple floors, spot types, vehicle types. `park(vehicle)` finds nearest available spot matching vehicle size, creates Ticket. `unpark(ticketId)` calculates fee by duration, frees spot. Fee via Strategy pattern. Demo: park 2 cars, park motorcycle, unpark one, show available spots. Most common machine coding + LLD crossover problem.

50. How do you design a snake game in machine coding?

Grid-based game: snake moves continuously, player changes direction, food appears randomly, snake grows on eating food, game over on wall/self collision. Classes: `Board`, `Snake` (body as linked list or deque), `Game` (game loop, direction input). Use Queue for snake body (add head, remove tail unless eating). Timer or turn-based for simplicity.